

Report Laboratorio

Nicolò Carbone 2139688

Gabriele Di Martino 2147973

Gabriele Guido 2144286

Obiettivo del progetto

Implementazione di un Parser di siti web per estrazione live di testo importante e di un meccanismo di valutazione del risultato tramite Gold Standard, supportando un database e un modello LLM per un ulteriore riscontro.

Organizzazione del codice

L'architettura del sistema segue un approccio modulare e containerizzato. Di seguito viene illustrata la gerarchia completa dei file di progetto:

```
./PROGETTO/
├── backend
│   ├── Dockerfile
│   ├── requirements.txt
│   └── src
│       ├── cleaner.py
│       ├── mariadb_data
│       ├── parser_nbcnews.py
│       ├── parser_uefa.py
│       ├── parser_weather.py
│       ├── parser_wikipedia.py
│       ├── populate_db.py
│       ├── pydantic_models.py
│       ├── server.py
│       ├── stats.txt
│       └── token_compare.py
├── docker-compose.yaml
├── domains.json
├── frontend
│   ├── Dockerfile
│   ├── requirements.txt
│   ├── src
│   │   └── frontend.py
│   ├── static
│   │   ├── gs_style.css
│   │   ├── home_style.css
│   │   ├── parser_evaluation_style.css
│   │   └── stats_style.css
│   └── templates
│       ├── gold_standard_builder.html
│       ├── home.html
│       ├── parser_evaluation.html
│       └── stats.html
├── gs_data
│   ├── nbcnews_gs.json
│   ├── uefa_gs.json
│   ├── weather_gs.json
│   └── wikipedia_gs.json
├── mariadb_data
│   ├── init.sql
│   └── ollama_data
└── report.pdf
```

L'infrastruttura dei container è suddivisa in quattro (più uno) servizi orchestrati con DockerCompose:

Backend:8003 Il core del sistema risiede nel modulo `server.py`, basato sul framework **FastAPI**, che gestisce le richieste HTTP e delega l'estrazione ai parser specifici. Il server implementa i metodi di risposta utilizzando classi di **Input/Output** (Pydantic models), definite in un

modulo apposito e separato, per garantire la validazione dei dati. Il processamento è supportato da moduli ausiliari contenenti metodi statici:

- **Cleaner**: Esegue il post-processing del markdown tramite **regex** per la normalizzazione del testo e l'estrazione di metadati (titolo e dominio) partendo da un url.
- **TokenCompare**: Gestisce la tokenizzazione e il calcolo delle statistiche comparative.

I moduli relativi ai **parser** incapsulano le configurazioni di Crawl4AI e restituiscono sia il testo elaborato che l'HTML sorgente. Per l'estrazione abbiamo impostato delle configurazioni che sfruttano i tag **HTML** e i selettori **CSS** per includere/escludere le info della pagina. Nel caso di `weather.com` è stato necessario aggiungere uno snippet in *JavaScript* per aggiungere degli spazi tra i tag affinché le info come "ora-temperatura" risultassero separate e non come un singolo token.

Frontend:8004 Il frontend invia richieste HTTP al server di backend con il modulo `request` e mostra le informazioni in modo dinamico tramite `jinj2`. La web UI si compone di 4 pagine: una home page e 3 pagine operative. Attraverso le tre pagine è possibile valutare testi parsati live/local, aggiungere al database altri gold standard ed infine visualizzare le statistiche globali per i domini presenti in database che riguardano numero di entry e metriche per la valutazione del testo parsato.

Database:3306 Il servizio monta un volume su cui viene implementato un database MariaDB inizializzato con un file `init.sql` in cui, oltre alla definizione delle tabelle, si crea un utente per permettere al backend di accedervi con la propria password, limitando l'accesso al solo database dedicato.

Ollama (e Ollamapull):11434 Il servizio di Ollama usa un volume per il modello di LLM (**Llama3.2**) e si appoggia a un ulteriore servizio dedicato al "pull" del modello, da cui dipenderà inoltre la costruzione degli altri servizi. I servizi seguono un ordine di avvio preciso:

ollama → ollamapull → DB → backend → frontend

Valutazione dei Parser

La valutazione dell'efficacia si basa sul confronto sistematico tra gli insiemi di token estratti e il Gold Standard (GS).

Metodologia di confronto

Il testo estratto manualmente (*gold text*) viene confrontato con l'output del parser. Entrambi vengono normalizzati in *lowercase* e suddivisi in token per popolare gli insiemi S_{gs} (gold standard) e S_p (parsati).

Metriche di performance

Vengono calcolate tre metriche nell'intervallo $[0, 1]$:

- **Precision** (capacità di escludere rumore):

$$\frac{|S_{gs} \cap S_p|}{|S_p|}$$

- **Recall** (capacità di recuperare token corretti):

$$\frac{|S_{gs} \cap S_p|}{|S_{gs}|}$$

- **F1** (media armonica tra le prime due):

$$\frac{2 \cdot Precision \cdot Recall}{Precision + Recall}$$

Valutazione dei Judge

La valutazione con LLM usa il modello di Ollama *"llama3.2:3b"* interrogato con un prompt configurato in modalità chat in modo da distinguere in maniera rigida i ruoli: il contesto con *system* e la richiesta come *user*. Con la tecnica del **Few-Shot Prompting** siamo riusciti a fornire delle regole ben precise in modo schematizzato: si forniscono al modello alcuni esempi di input e output desiderato prima della domanda reale, con maggior enfasi su eccezioni e casi limite. Si è poi reso il contesto più generale in modo da renderlo valido per tutti i domini. La richiesta *user* si limita a fornire all'LLM i due testi per effettuare la valutazione in termini di *score* (intero tra 1 a 5) e *feedback*. Il modello impara il pattern e lo applica al nuovo input.

Risultati ottenuti

Nella tabella seguente sono riportate le statistiche medie ottenute dal Gold Standard di riferimento:

Dominio	Precision	Recall	F1	Score
NBC News	0.89	0.99	0.94	4.9
Wikipedia (en)	0.96	0.91	0.93	5.0
UEFA	0.82	0.95	0.87	5.0
Weather	0.84	0.91	0.87	3.6

Table 1: Performance medie dei parser rispetto al Gold Standard.

Organizzazione del DB

Il backend ha accesso al database *lab_db* composto da tre tabelle:

- *web_resources(url, domain, title, html_text)*: risorse principali del database;
- *gold_standard(url, gold_text)*: gold text di una risorsa web;
- *stats(url, prec, rec, f1, score)*: statistiche per ogni url ;

La prima tabella ha l'attributo **url** che è anche un riferimento per gli altri url delle altre tabelle, con una politica di *eliminazione a cascata*.